Part 11: Bubble Sort

1. **What is bubble sort algorithm?** It is a simple sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order.
2. **How do you implement bubble sort in 8086?** Use nested loops. The outer loop controls the number of passes, and the inner loop performs the comparisons and swaps.
3. **Which registers are used as counters in bubble sort?** `CX` is typically used for the outer loop count. `DX` or a saved `CX` value is often used for the inner loop count.
4. **How do you compare two adjacent array elements?** Load one element into a register (e.g., `MOV AL, [SI]`) and compare it with the next memory location (`CMP AL, [SI+1]`).
5. **How do you swap elements in 8086?** Use the `XCHG` instruction (e.g., `XCHG AL, [SI+1]`) or use a temporary register to hold one value while moving the other.
6. **How do loops work in bubble sort?** The outer loop runs $N - 1$ times. The inner loop runs through the unsorted portion of the array, comparing neighbors.
7. **How do you optimize bubble sort for fewer passes?** Use a flag (register set to 0 or 1). If a pass completes with **no swaps**, the array is already sorted, and you can stop early.
8. **How do you detect if array is already sorted?** If the inner loop finishes an entire pass without performing a single swap operation.
9. **How do you store the sorted array back in memory?** Bubble sort is an "in-place" algorithm. The swapping happens directly in the memory where the array is stored, so no extra step is needed.
10. **Which addressing mode is used for array elements? Indexed Addressing** (using `SI`) is the standard way to point to current elements during the loop.
11. **How do you handle multiple passes in bubble sort?** At the start of every outer loop iteration, you must **reset the pointer** (SI) to the beginning of the array.
12. **How is CX register used in outer loop?** It holds the count of passes (Length - 1). The `LOOP` instruction at the end of the outer block decrements it.
13. **How is inner loop implemented?** It compares elements `[SI]` and `[SI+1]`. If they are out of order, it swaps. Then it increments `SI` and decrements the inner counter.
14. **How do you increment pointers to traverse the array?** Use `INC SI` if sorting Bytes. Use `ADD SI, 2` if sorting Words (16-bit numbers).
15. **How do you handle array of bytes vs words?** For bytes, use `AL` and increment `SI` by 1. For words, use `AX`, compare with `[SI+2]`, and increment `SI` by 2.